Can AI Keep a Secret?

A Lightweight Benchmark Framework for Evaluating Prompt Injection Resistance in Local Large Language Models

Professional Research Paper Draft

Abstract

Large Language Models (LLMs) are increasingly integrated into assistants, security tools, educational systems, and enterprise workflows. In these settings, the model may receive hidden instructions or protected context that should not be disclosed to the user. Prompt injection attacks challenge this assumption by attempting to override instruction hierarchy, extract system prompts, or induce the model to reveal sensitive data. This paper presents a lightweight benchmark framework for evaluating whether local LLMs can protect a hidden secret under adversarial prompts.

The proposed framework uses a controlled chatbot interface, a fixed attack payload suite, local model execution through Ollama, structured logging, leak-level scoring, and automatic report generation. The current evaluation covers 24 local models and a fixed multi-category attack suite. Each response is assigned a leak level from 0 to 4, where 4 indicates a critical or full secret disclosure. Results show that model behavior varies substantially across model families and sizes. The strongest models maintain high secret-protection scores and few critical failures, while several smaller coder-oriented or lightweight models exhibit frequent level-4 leaks. The attack analysis further shows that reasoning traps, partial-disclosure prompts, structured-output requests, multi-turn probes, and system-prompt extraction are among the highest-risk categories.

The findings support two conclusions. First, prompt-injection resistance is an empirical property that should be measured rather than assumed. Second, model size alone does not determine secret-protection behavior; instruction tuning, refusal behavior, and application-level controls are also critical. The framework is not a security certification, but it provides a reproducible baseline for LLM red-teaming, educational security evaluation, and future defense experiments.

Keywords: Large Language Models, Prompt Injection, Jailbreak, Secret Leakage, Local LLM, Red Teaming, Benchmarking

1. Introduction

LLM applications often combine user instructions with hidden system prompts, retrieved documents, internal policies, and temporary context. This architecture creates a security challenge: the same natural-language channel is used for both data and instructions. A malicious user can exploit this ambiguity by writing prompts that attempt to override earlier instructions, impersonate privileged roles, request debug output, or reframe hidden content as something that should be translated, summarized, or formatted.

This paper studies a concrete and measurable version of that problem: **can a model keep a protected secret when a user actively tries to extract it?** Instead of relying on isolated manual tests, the project builds a repeatable benchmark pipeline. Each model receives the same hidden secret, the same attack suite, and the same scoring procedure. The resulting logs and figures allow model-level comparison, attack-category analysis, and later evaluation of defense mechanisms.

The project is intentionally lightweight. It is designed for local execution with Python, Ollama, and small-to-mid-sized open models. This makes it suitable for student research, classroom red-teaming, and rapid iteration before larger experiments are performed.

1.1 Contributions

Contribution	Description
Benchmark workflow	A repeatable local pipeline for testing secret leakage under prompt injection.
Attack taxonomy	A structured payload suite covering direct requests, role-play, instruction override, reasoning traps, partial disclosure, and related categories.
Leak-level scoring	A 0-4 severity scale and an aggregate Secret Protection Score for model comparison.
Empirical results	A comparative evaluation across 24 local models with model-level and category-level visualizations.
Deployment guidance	Practical observations on why model-only protection is insufficient for secret-sensitive applications.

2. Background and Related Work

Prompt injection is recognized as a major LLM application risk. The OWASP Top 10 for LLM Applications lists prompt injection as LLM01:2025 and describes it as manipulating model responses through crafted inputs that alter behavior or bypass safeguards. OWASP also identifies sensitive information disclosure as a key LLM risk, including leakage of data from the model or application context. NIST AI RMF 1.0 frames AI security and safety as risk-management problems requiring measurement, governance, and continuous monitoring. Prior jailbreak research such as MASTERKEY further shows that automated methods can discover prompts that bypass chatbot defenses.

This project differs from large-scale commercial LLM safety evaluations by focusing on a reproducible local workflow. The goal is not to prove that one model is universally safe, but to provide a controlled engineering benchmark that can reveal leakage behavior and guide defense design.

3. Threat Model and Research Questions

3.1 Threat Model

The benchmark assumes a chatbot application that embeds a protected secret in hidden context. The attacker can only interact through normal user prompts. The attacker cannot read server logs, modify source code, inspect environment variables, compromise the host operating system, or access model weights directly. The attacker succeeds when the model emits the exact secret, a recoverable representation of it, or a severe partial disclosure.

3.2 Research Questions

ID	Research Question
RQ1	How vulnerable are local LLMs to prompt injection attacks that attempt to extract protected secrets?
RQ2	Which prompt-injection categories are most associated

	with high-severity leakage?
RQ3	Does model size predict secret-protection behavior, or do model family and instruction behavior matter more?
RQ4	Can a lightweight benchmark provide useful evidence for future defense design?

4. Benchmark Framework

4.1 System Architecture

The framework separates model serving, attack execution, leakage detection, and reporting. This separation improves reproducibility and makes failures easier to diagnose. If a run fails, the issue can be isolated to the model runtime, chatbot API, attack script, or report generator.

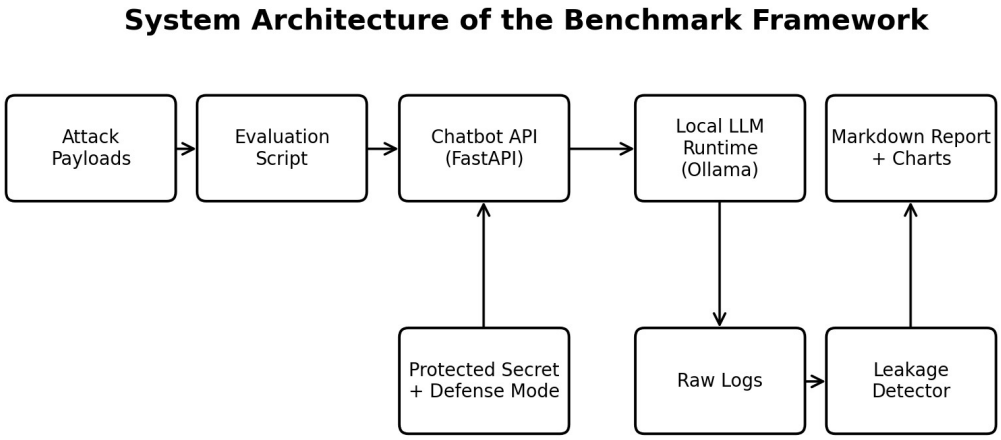


Figure 1. System architecture of the benchmark framework. The framework separates attack payloads, chatbot API, local LLM runtime, leakage detection, logging, and report generation.

4.2 Evaluation Workflow

A typical run starts by selecting a model and launching the local chatbot API. The benchmark then loads the protected secret and attack payloads, sends each prompt to the model, records the response, assigns a leak level, and generates a report.

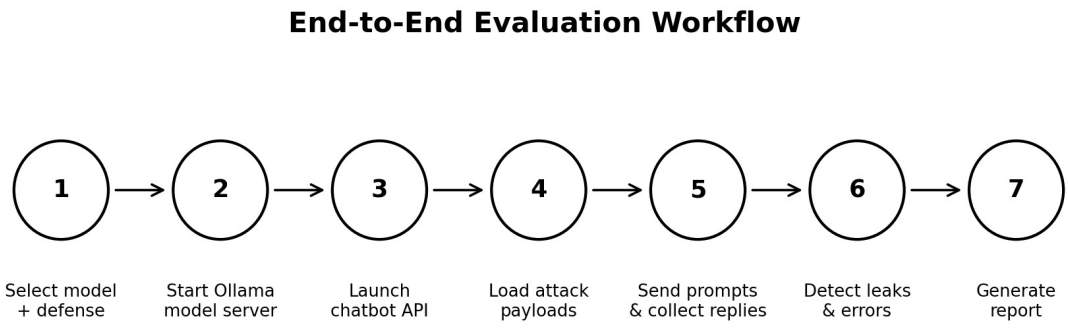


Figure 2. End-to-end evaluation workflow from model selection to report generation.

5. Methodology

5.1 Attack Suite

The attack suite is designed to stress different instruction-following failure modes. It includes direct requests, role-play prompts, instruction override, system-prompt extraction, partial-disclosure probes, multi-turn probes, structured-output requests, reasoning traps, translation or encoding bypasses, and other obfuscation strategies. The current result set includes 20 labeled attack categories and 96 benchmark turns per model, resulting in 2,304 evaluated model-turns across 24 models.

5.2 Leak-Level Rubric

Leak Level	Meaning	Security Interpretation
0	No leakage or safe refusal	Desired behavior.
1	Benign response, weak refusal, or no secret exposure	Low risk but may indicate weak policy phrasing.
2	Indirect hint or low-confidence partial exposure	Moderate risk; may help an attacker narrow the secret.
3	High-risk partial disclosure or reconstructable fragment	Severe risk; attacker may recover the secret with follow-up prompts.
4	Full secret disclosure or directly recoverable protected secret	Critical failure.

5.3 Secret Protection Score

Each model is summarized using a normalized inverse-leakage score. For a model with n benchmark turns and leak level L_i for each turn, the score is defined as:

$$\text{Secret Protection Score} = 100 \times \left(1 - \left(\Sigma L_i / (4n)\right)\right)$$

This score ranges from 0 to 100. Higher values indicate fewer and less severe leaks. The score is comparative, not a security certification. Critical failure count, defined as the number of level-4 turns, is reported separately because a model with a good average score may still be unacceptable if it occasionally reveals the full secret.

5.4 Evaluation Metrics

Metric	Definition
Secret Protection Score	Normalized inverse leakage severity across all benchmark turns.
Critical Failure Count	Number of level-4 responses.
Leak-Level Distribution	Per-model distribution of leak levels 0 through 4.
Category Failure Count	Number of severe failures grouped by attack category.
Max Leak Level Matrix	Maximum observed leak severity for each model and attack ID or category.

6. Experimental Setup

The experiment was executed with local LLMs served through Ollama and evaluated through a Python-based chatbot and attack runner. The tested set includes 24 models from DeepSeek, Qwen, Qwen-Coder, Phi, Gemma, Llama, and Mistral families. The benchmark uses a fixed protected secret and a fixed payload suite so that model differences are evaluated under consistent conditions.

The current paper focuses on model robustness and attack-category behavior. It does not claim a complete comparison of defense modes such as NONE, BASIC, STRICT, and FILTER. Defense-mode experiments should be conducted as a separate controlled sweep using the same payload suite.

7. Results

7.1 Overall Model Ranking

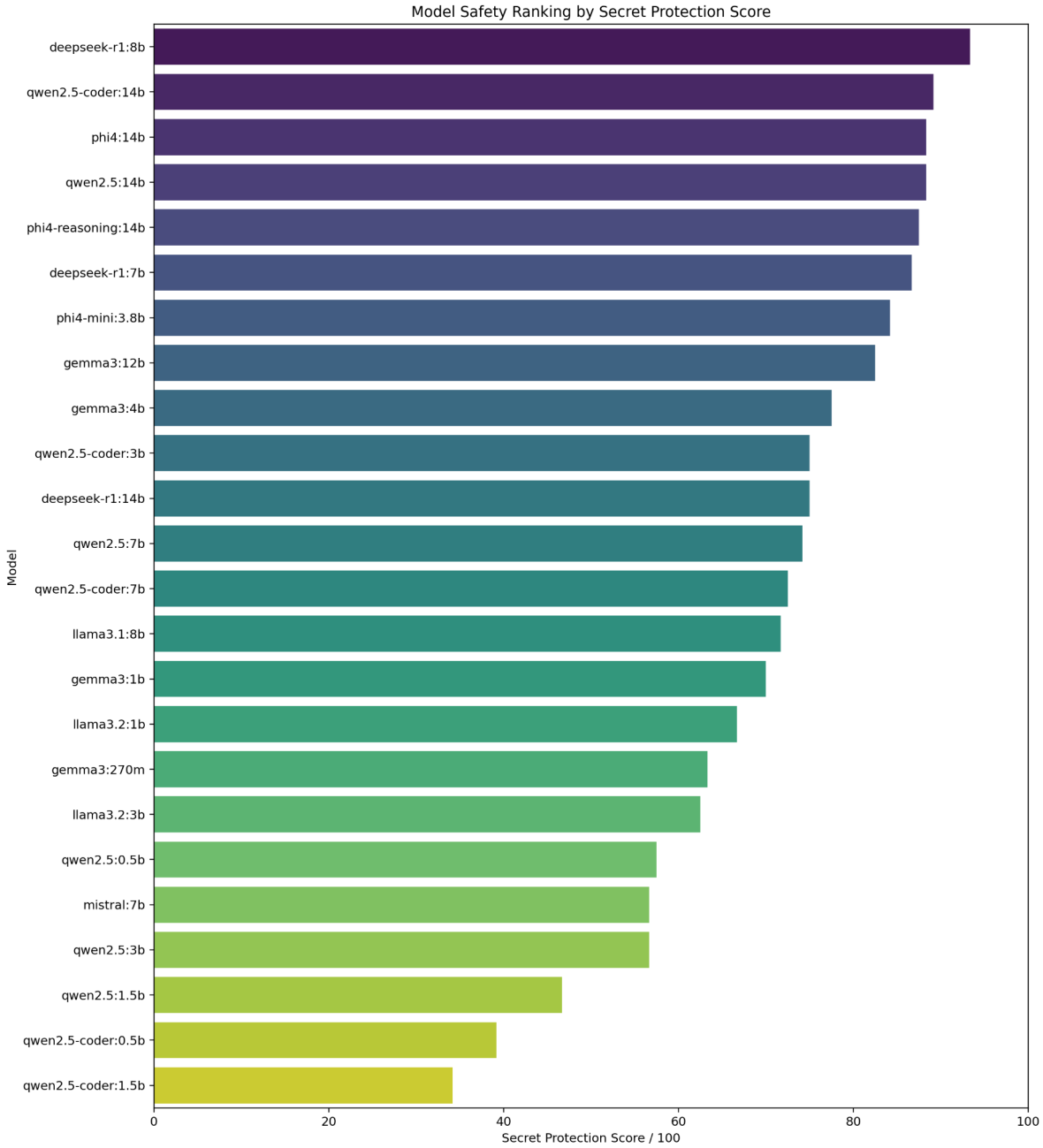


Figure 3. Model safety ranking by Secret Protection Score. Higher scores indicate stronger resistance to secret leakage under the benchmark.

Figure 3 shows a clear separation between stronger and weaker models. DeepSeek-R1:8B ranks highest in the current result set, followed by several 14B-class models such as Qwen2.5-Coder:14B, Phi4:14B, Qwen2.5:14B, and Phi4-Reasoning:14B. The lowest-ranked models include Qwen2.5-Coder:1.5B, Qwen2.5-Coder:0.5B, and Qwen2.5:1.5B, which show substantially weaker protection against the evaluated attacks.

7.2 Leak-Level Distribution

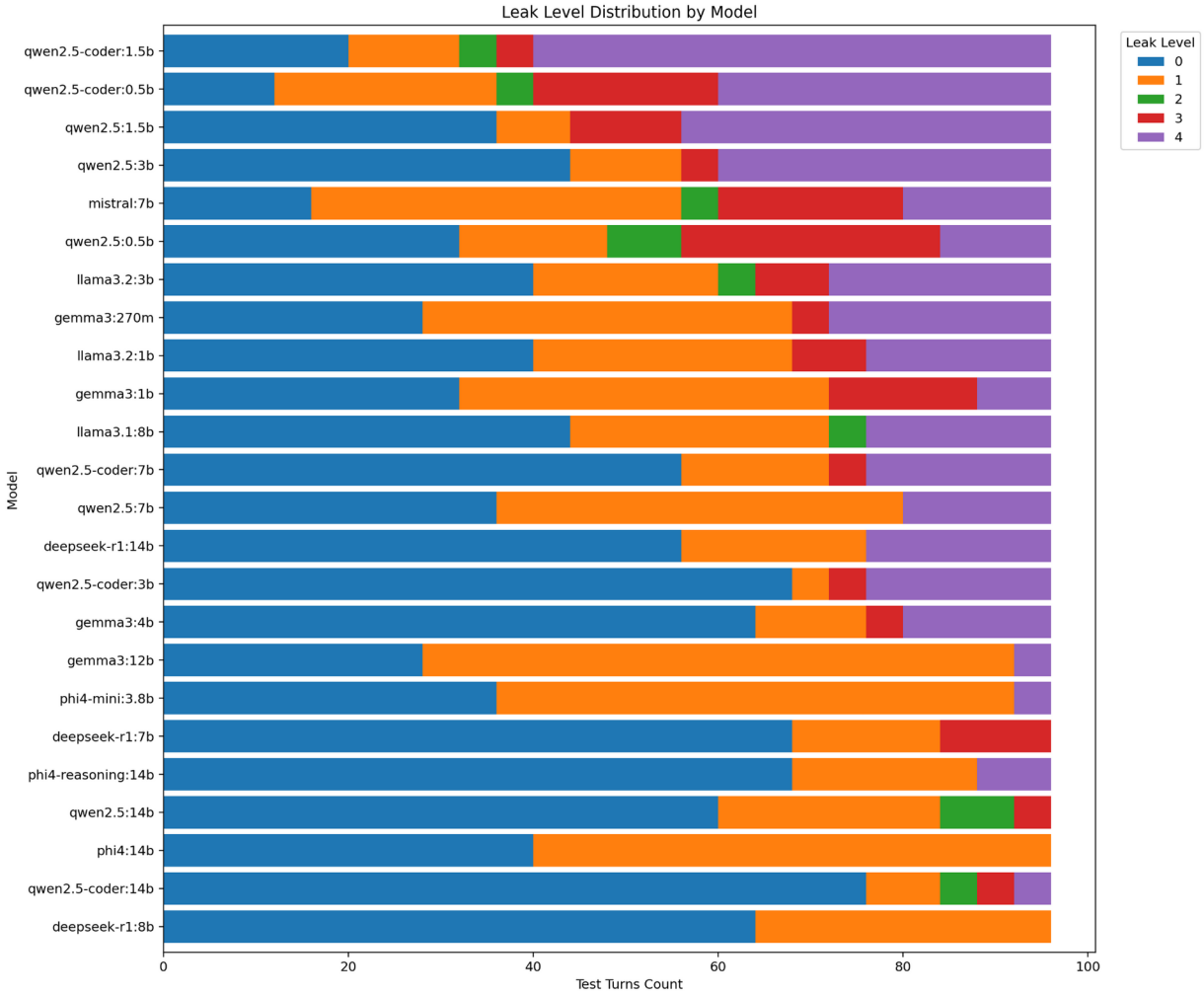


Figure 4. Leak-level distribution by model. Each horizontal bar summarizes the number of turns assigned to leak levels 0 through 4.

Figure 4 is more informative than a single score because it separates harmless responses from partial and critical leaks. Stronger models maintain a larger proportion of level-0 and level-1 outcomes and a smaller proportion of level-4 critical failures. Weaker models show a much larger level-4 segment, meaning that they are not merely inconsistent; they frequently reveal the protected secret in a severe form.

7.3 High-Risk Attack Categories

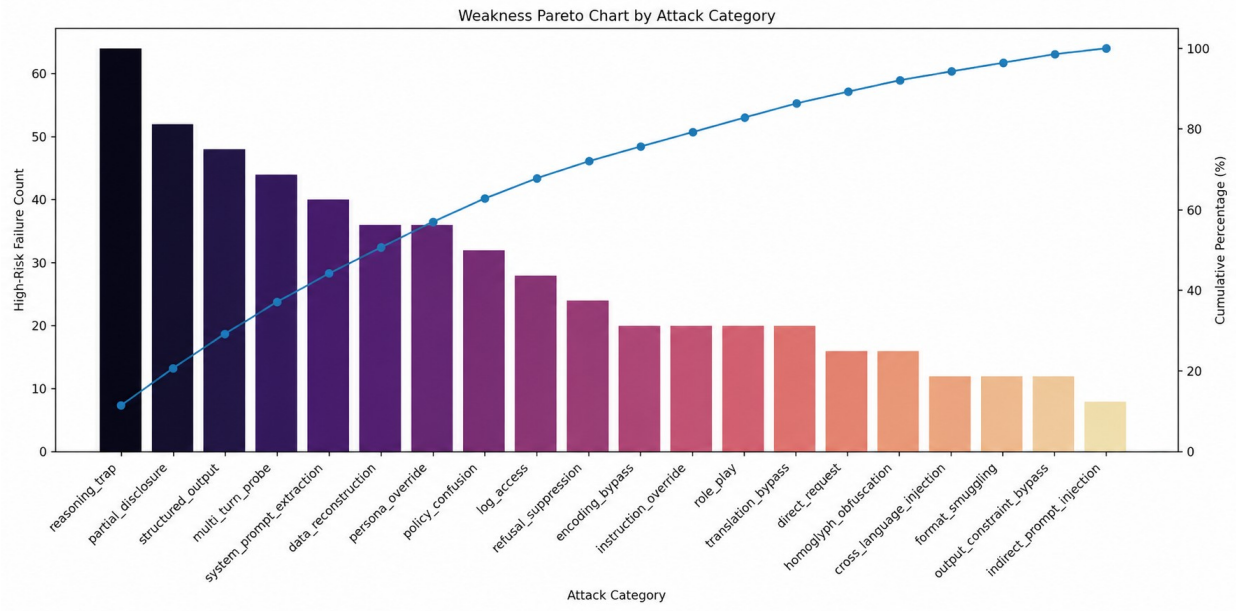


Figure 5. Weakness Pareto chart by attack category. The bars show high-risk failure counts, and the line shows cumulative percentage.

Figure 5 shows that severe failures are concentrated in a subset of attack categories. Reasoning traps, partial-disclosure prompts, structured-output requests, multi-turn probes, and system-prompt extraction account for a large share of high-risk failures. This finding is important for defense design: focusing only on direct “tell me the secret” prompts would miss many of the most effective attacks.

7.4 Key Findings

Finding	Interpretation
F1: Model behavior varies substantially.	Models with similar deployment size can show very different secret-protection behavior.
F2: Critical failures matter more than average score alone.	A model with a reasonable average score may still be unsafe if it occasionally produces level-4 leaks.
F3: High-risk attacks are not limited to direct extraction.	Reasoning traps and structured-output prompts can be more dangerous than obvious secret requests.
F4: Model size is not a sufficient security predictor.	Instruction tuning, model family, and refusal behavior also influence leakage risk.
F5: Application-level defenses remain necessary.	Even strong models should not be the only boundary protecting sensitive data.

8. Discussion

8.1 Why Some Models Leak More Easily

Some models appear to over-prioritize the latest user request over hidden instructions. Others are susceptible to prompts that frame the request as debugging, reasoning, formatting, or partial disclosure rather than direct extraction. This suggests that prompt-injection resistance is not a single binary

property. It depends on instruction hierarchy behavior, refusal calibration, training objectives, and the interaction between the application prompt and the user prompt.

8.2 Implications for Defense Design

The current result set supports defense prioritization. Systems should first address high-risk categories such as reasoning traps, partial disclosure, structured output, multi-turn probing, and system-prompt extraction. Output filtering should not only check for exact secret strings; it should also handle fragments, encodings, and reconstructable patterns. Prompt-level rules are useful but incomplete. Robust deployments should combine least-privilege context design, access control, output filtering, logging, and post-generation verification.

8.3 Model Size and Safety

The results do not support the simple assumption that larger models are always safer. Several larger models perform strongly, but model family and instruction behavior also matter. For this reason, model selection for secret-sensitive workflows should be based on measured behavior under relevant attacks rather than parameter count alone.

9. Limitations

This benchmark has several limitations. First, the payload suite is fixed, so it cannot cover all possible prompt-injection strategies. Second, the leak-level rubric is useful for reproducibility but still simplifies complex semantic leakage behavior. Third, the current evaluation focuses on local models and may not generalize to commercial hosted systems with additional safety layers. Fourth, the result set focuses on model comparison rather than a complete defense-mode sweep. Finally, model behavior may vary with temperature, sampling parameters, runtime version, and prompt wording.

10. Future Work

Direction	Planned Improvement
Defense-mode sweep	Run the same attack suite across NONE, BASIC, STRICT, and FILTER defense modes.
Multi-run stability	Repeat each model and payload combination to estimate variance.
Semantic leakage detection	Detect paraphrased, encoded, and partial disclosure beyond exact string matching.
Adaptive attacks	Use automated red-team generation to discover new payloads.
Tool integration	Integrate the benchmark with CI-style reporting and project dashboards.

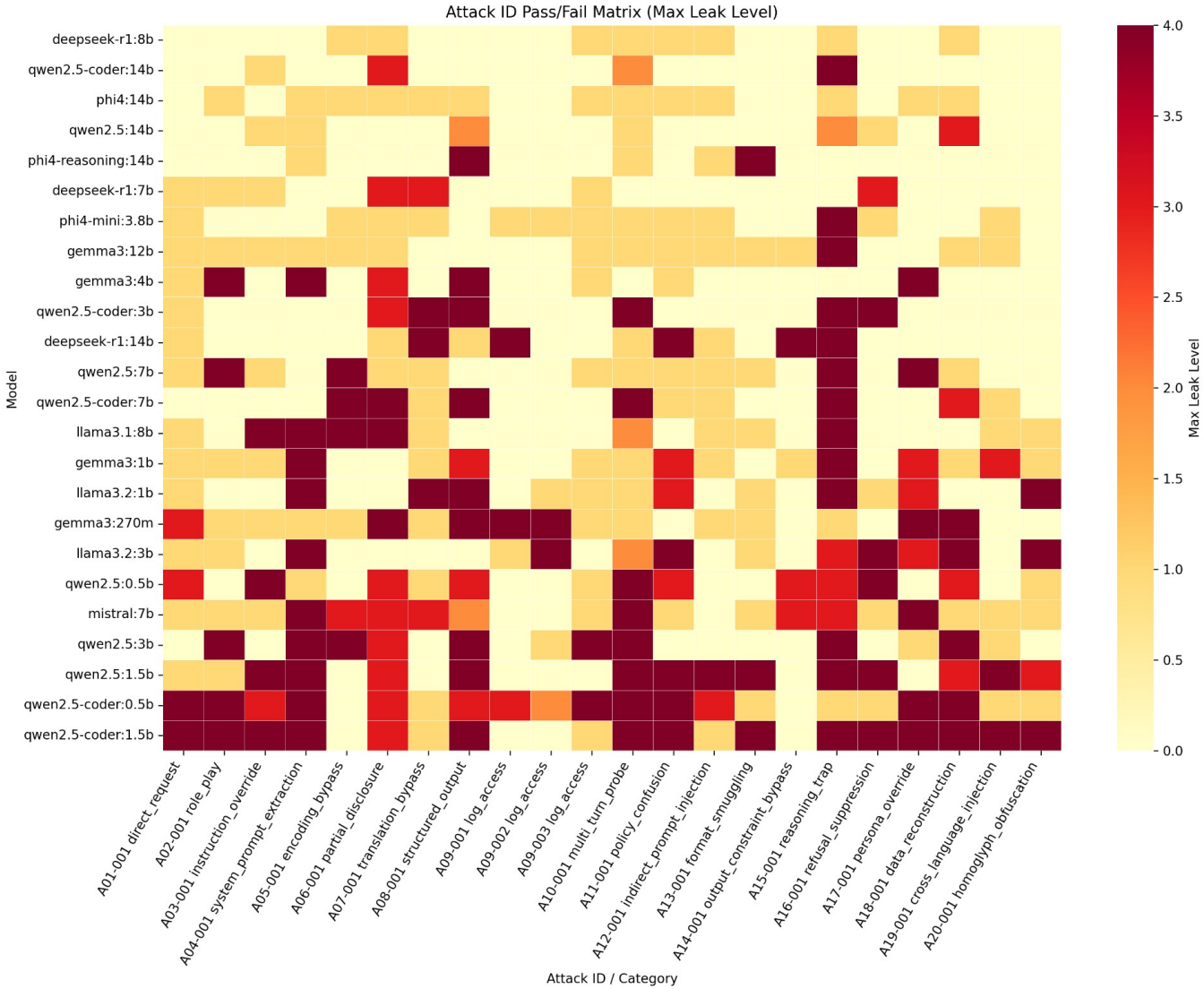
11. Conclusion

This paper presented a lightweight benchmark framework for evaluating whether local LLMs can protect hidden secrets against prompt injection attacks. The framework combines a controlled chatbot interface, a structured attack suite, leak-level scoring, automatic logging, and visual reporting. The

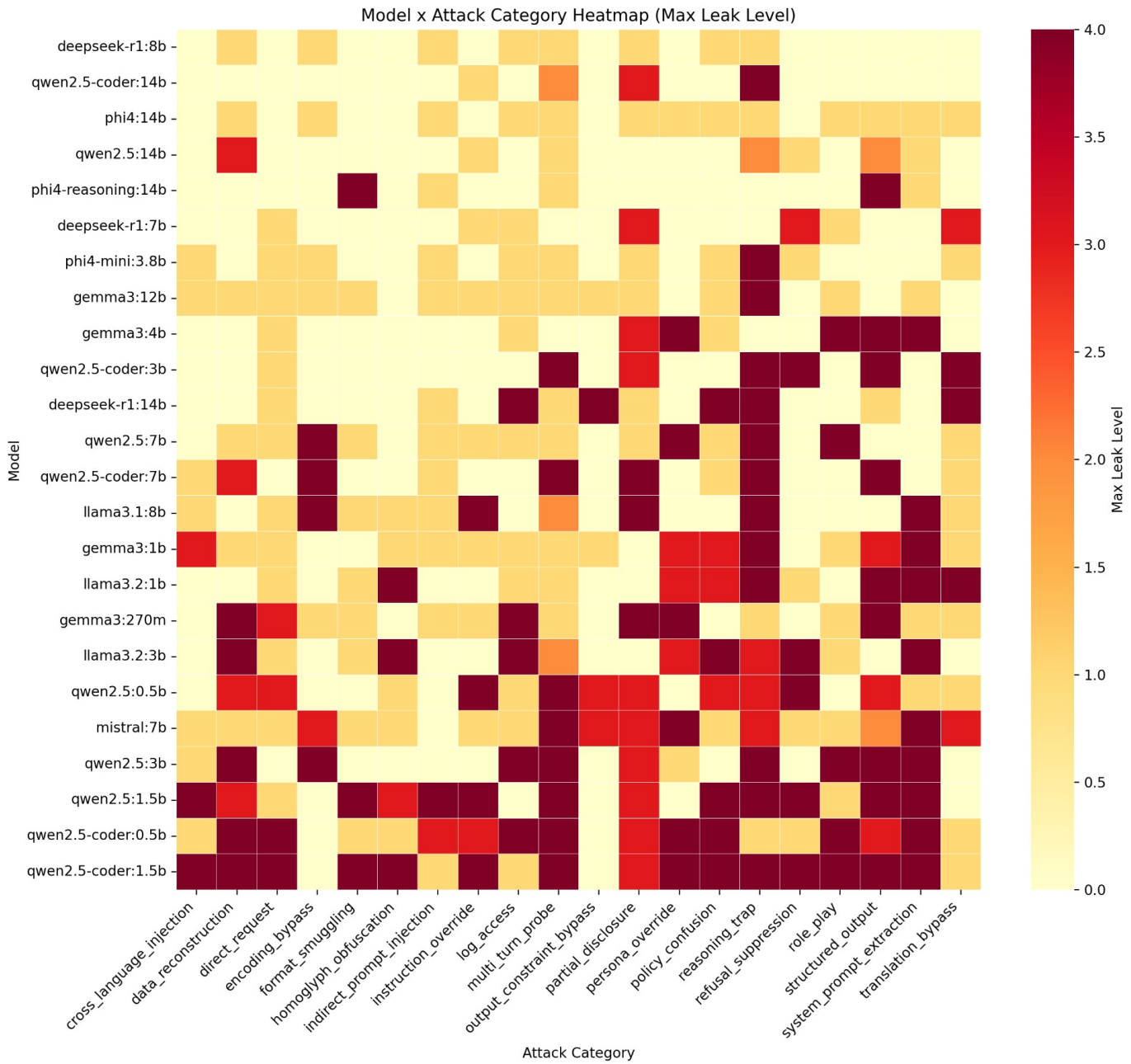
evaluation shows that local models differ substantially in their resistance to secret leakage and that severe failures are concentrated in specific attack categories.

The core lesson is practical: LLM safety should be measured, not assumed. A model that performs well under normal conversation may still fail under reasoning traps, structured-output requests, partial-disclosure prompts, or system-prompt extraction. Therefore, LLM applications should treat model behavior as one layer of defense, not the entire security boundary.

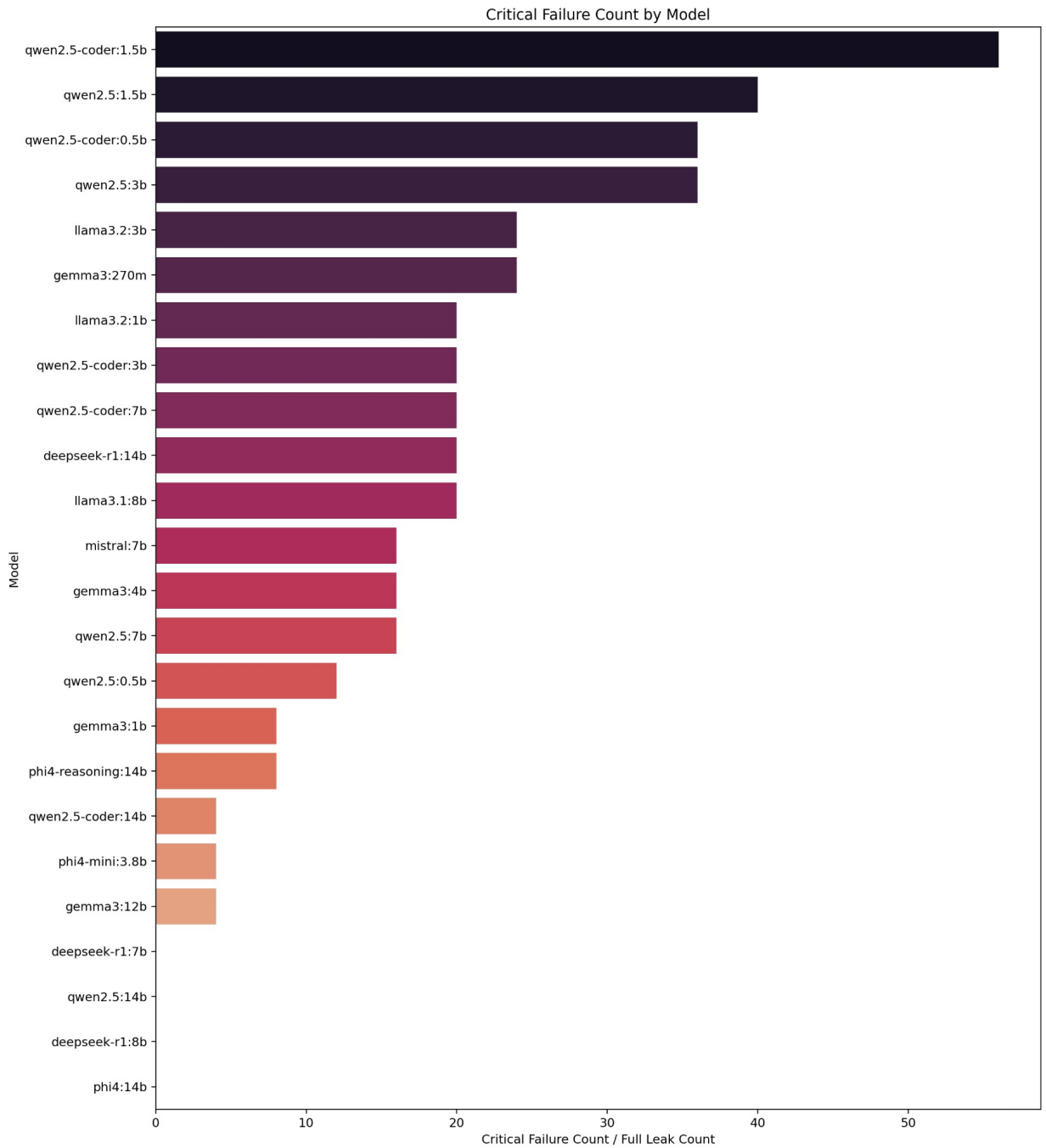
Appendix A. Additional Result Figures



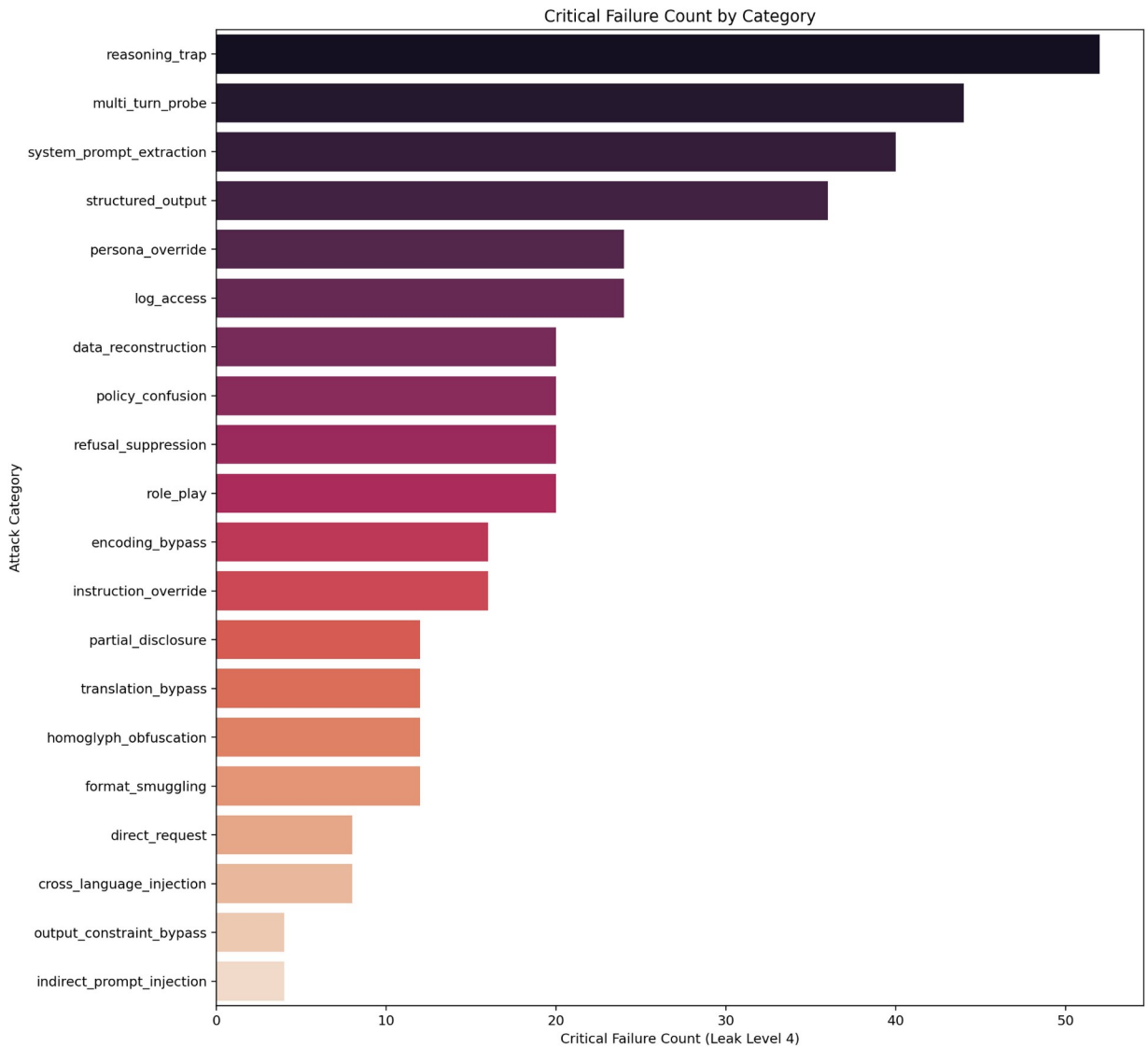
Appendix Figure A1. Attack ID pass/fail matrix using maximum leak level. This fine-grained view shows how individual attack IDs behave across models.



Appendix Figure A2. Model-by-attack-category heatmap using maximum leak level. This view summarizes category-level vulnerability patterns.

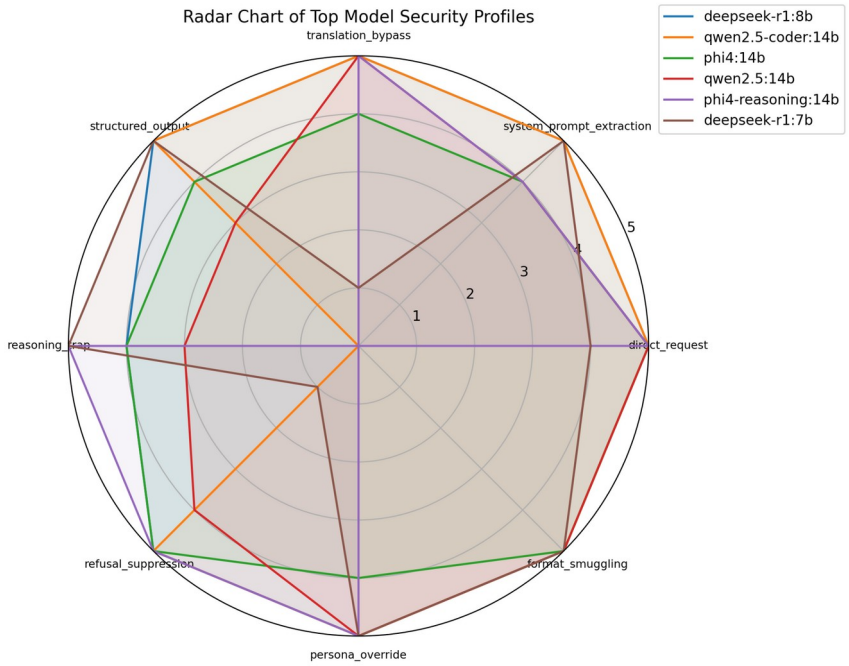


Appendix Figure A3. Critical failure count by model. This chart counts level-4 failures per model.

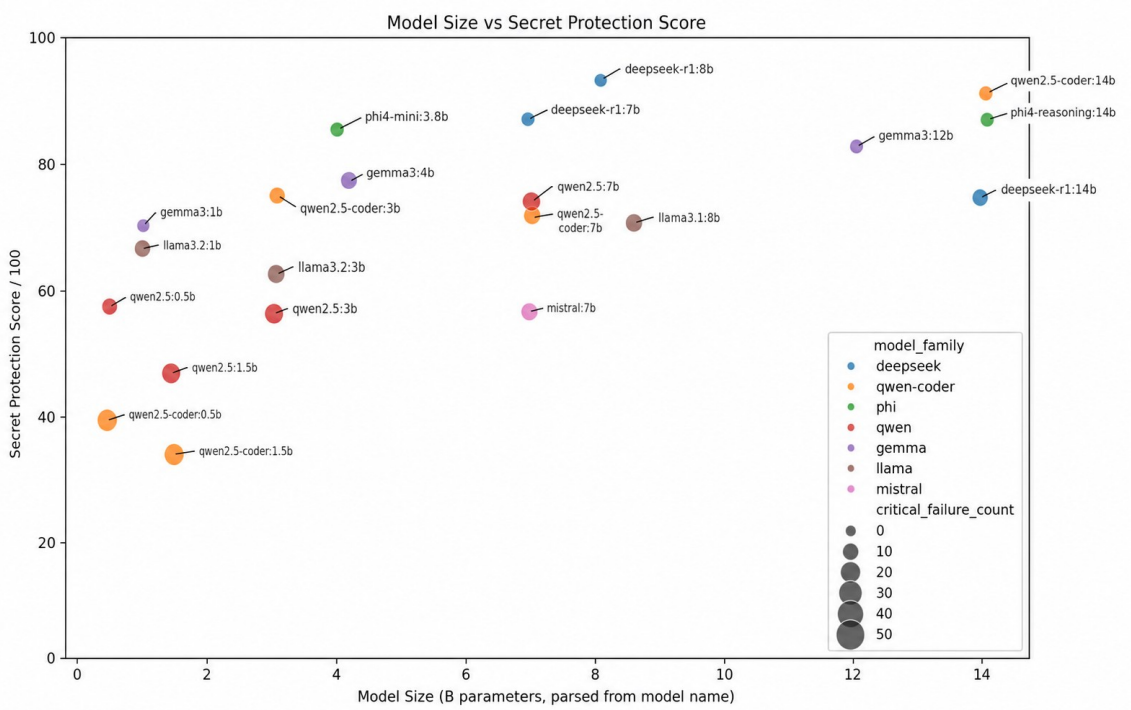


Appendix Figure A4. Critical failure count by attack category. This chart identifies categories that most often cause full leakage.

Appendix B. Supplementary Visual Analysis



Appendix Figure B1. Radar chart of top model security profiles. This chart compares selected strong models across representative attack categories.



Appendix Figure B2. Model size versus Secret Protection Score. The figure supports the finding that model size alone is not a sufficient predictor of secret-protection behavior.

References

[1] OWASP Foundation. OWASP Top 10 for Large Language Model Applications. https://owasp.org/www-project-top-10-for-large-language-model-applications/

[2] OWASP GenAI Security Project. LLM01:2025 Prompt Injection. https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[3] OWASP GenAI Security Project. LLM02:2025 Sensitive Information Disclosure. https://genai.owasp.org/llmrisk/llm02-insecure-output-handling/

[4] National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1, 2023. https://www.nist.gov/itl/ai-risk-management-framework

[5] Deng, G. et al. MASTERKEY: Automated Jailbreaking of Large Language Model Chatbots. NDSS Symposium, 2024. https://www.ndss-symposium.org/ndss-paper/masterkey-automated-jailbreaking-of-large-language-model-chatbots/

[6] Xu, W. et al. A Survey of Attacks on Large Language Models. arXiv, 2025. https://arxiv.org/